

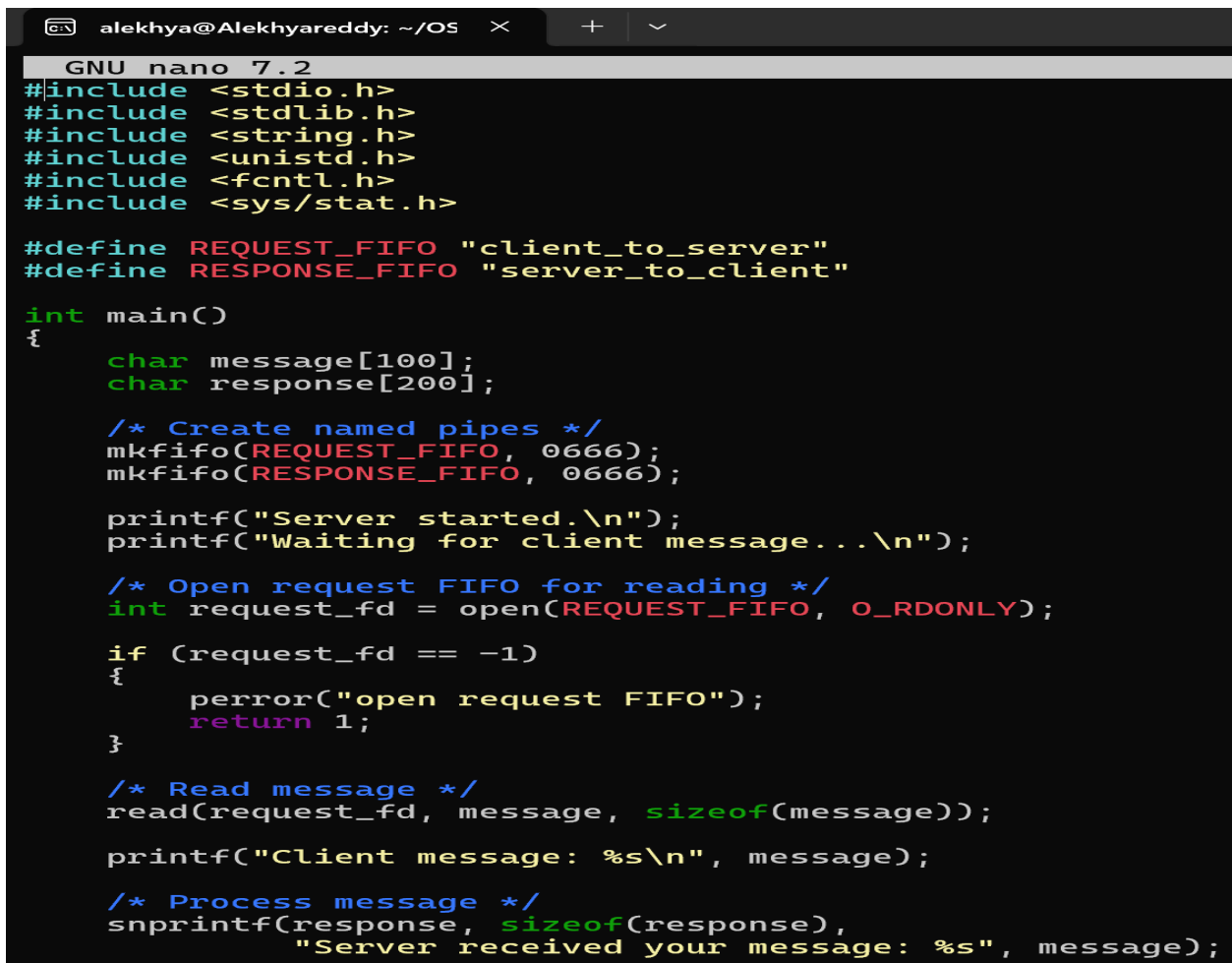
WEEK - 6 PRACTICAL

1 . Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Aim :

To implement client-server communication using Named Pipes (FIFOs), where clients send messages to a server and the server processes and responds to them.

Program In Terminal 1 :



```
alekhya@Alekhyareddy: ~/OS × + v
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define REQUEST_FIFO "client_to_server"
#define RESPONSE_FIFO "server_to_client"

int main()
{
    char message[100];
    char response[200];

    /* Create named pipes */
    mkfifo(REQUEST_FIFO, 0666);
    mkfifo(RESPONSE_FIFO, 0666);

    printf("Server started.\n");
    printf("Waiting for client message...\n");

    /* Open request FIFO for reading */
    int request_fd = open(REQUEST_FIFO, O_RDONLY);

    if (request_fd == -1)
    {
        perror("open request FIFO");
        return 1;
    }

    /* Read message */
    read(request_fd, message, sizeof(message));

    printf("Client message: %s\n", message);

    /* Process message */
    snprintf(response, sizeof(response),
             "Server received your message: %s", message);
```

```
if (request_fd == -1)
{
    perror("open request FIFO");
    return 1;
}

/* Read message */
read(request_fd, message, sizeof(message));

printf("Client message: %s\n", message);

/* Process message */
snprintf(response, sizeof(response),
          "Server received your message: %s", message);

close(request_fd);

/* Open response FIFO */
int response_fd = open(RESPONSE_FIFO, O_WRONLY);

if (response_fd == -1)
{
    perror("open response FIFO");
    return 1;
}

/* Send response */
write(response_fd, response, strlen(response) + 1);

printf("Response sent to client.\n");

close(response_fd);

printf("Server completed.\n");

return 0;
```

}

In terminal 2 :

```
alekhya@Alekhyareddy: ~/OS
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define REQUEST_FIFO "client_to_server"
#define RESPONSE_FIFO "server_to_client"

int main()
{
    char message[100];
    char response[200];

    printf("Enter message for server: ");
    fgets(message, sizeof(message), stdin);

    /* Remove newline */
    message[strcspn(message, "\n")] = '\0';

    /* Open request FIFO */
    int request_fd = open(REQUEST_FIFO, O_WRONLY);

    if (request_fd == -1)
    {
        perror("open request FIFO");
        return 1;
    }

    /* Send message */
    write(request_fd, message, strlen(message) + 1);

    close(request_fd);

    /* Open response FIFO */
    int response_fd = open(RESPONSE_FIFO, O_RDONLY);

    if (response_fd == -1)
    {
        perror("open response FIFO");
        return 1;
    }

    /* Read response */
    read(response_fd, response, sizeof(response));

    printf("Server response: %s\n", response);

    close(response_fd);

    return 0;
}
```

Outputs :

Terminal 1:

```
alekhya@Alekhyareddy: ~/OS × + v
alekhya@Alekhyareddy:~$ cd ~/OSSP
alekhya@Alekhyareddy:~/OSSP$ pwd
/home/alekhya/OSSP
alekhya@Alekhyareddy:~/OSSP$ nano server.c
alekhya@Alekhyareddy:~/OSSP$ gcc -Wall -Wextra server.c -o server
alekhya@Alekhyareddy:~/OSSP$ ./server
Server started.
Waiting for client message...
Client message: Hello Server
Response sent to client.
Server completed.
alekhya@Alekhyareddy:~/OSSP$ ls -l client_to_server server_to_client
prw-r--r-- 1 alekhya alekhya 0 Sep 18 03:52 client_to_server
prw-r--r-- 1 alekhya alekhya 0 Sep 18 03:52 server_to_client
```

Terminal 2:

```
alekhya@Alekhyareddy: ~/OS × + v
alekhya@Alekhyareddy:~$ cd ~/OSSP
alekhya@Alekhyareddy:~/OSSP$ nano client.c
alekhya@Alekhyareddy:~/OSSP$ gcc -Wall -Wextra client.c -o client
alekhya@Alekhyareddy:~/OSSP$ ./client
Enter message for server: Hello Server
Server response: Server received your message: Hello Server
alekhya@Alekhyareddy:~/OSSP$ |
```

2 . Create a POSIX signal handling program that captures:

- SIGINT
- SIGTERM
- SIGUSR1

Demonstrate asynchronous event handling and explain the role of signal handlers.

Aim : To implement POSIX signal handling using **SIGINT**, **SIGTERM**, and **SIGUSR1**, and demonstrate how signal handlers respond to asynchronous events.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int signal)
{
    if (signal == SIGINT)
    {
        printf("\nSIGINT received (Ctrl+C)\n");
    }
    else if (signal == SIGTERM)
    {
        printf("\nSIGTERM received\n");
    }
    else if (signal == SIGUSR1)
    {
        printf("\nSIGUSR1 received\n");
    }
}

int main()
{
    printf("Signal handling program started.\n");
    printf("Process ID: %d\n", getpid());

    /* Register signal handlers */
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("\nWaiting for signals...\n");

    while (1)
    {
        pause();
    }

    return 0;
}
```

Output :

Terminal 1 :

```
alekhya@Alekhyareddy: ~/OS  ×  +  ▾  
alekhya@Alekhyareddy:~$ cd ~/OSSP  
alekhya@Alekhyareddy:~/OSSP$ nano client.c  
alekhya@Alekhyareddy:~/OSSP$ gcc -Wall -Wextra client.c -o client  
alekhya@Alekhyareddy:~/OSSP$ ./client  
Enter message for server: Hello Server  
Server response: Server received your message: Hello Server  
alekhya@Alekhyareddy:~/OSSP$ nano signal_handler.c  
alekhya@Alekhyareddy:~/OSSP$ gcc -Wall -Wextra signal_handler.c -o signal_handler  
alekhya@Alekhyareddy:~/OSSP$ ./signal_handler  
Signal handling program started.  
Process ID: 825  
  
Waiting for signals...  
^C  
SIGINT received (Ctrl+C)  
  
SIGUSR1 received  
  
SIGTERM received  
Killed  
alekhya@Alekhyareddy:~/OSSP$ |
```

Terminal 2 :

```
alekhya@Alekhyareddy: ~  ×  +  ▾  
alekhya@Alekhyareddy:~$ kill -SIGUSR1 PID  
-bash: kill: PID: arguments must be process or job IDs  
alekhya@Alekhyareddy:~$ kill -SIGUSR1 4525  
-bash: kill: (4525) - No such process  
alekhya@Alekhyareddy:~$ kill -10 825  
alekhya@Alekhyareddy:~$ kill -SIGTERM 825  
alekhya@Alekhyareddy:~$ kill -KILL 825  
alekhya@Alekhyareddy:~$ kill %1kill %1  
-bash: kill: %1kill: no such job  
-bash: kill: %1: no such job  
alekhya@Alekhyareddy:~$ kill -9 PID  
-bash: kill: PID: arguments must be process or job IDs  
alekhya@Alekhyareddy:~$ kill -9 825  
-bash: kill: (825) - No such process  
alekhya@Alekhyareddy:~$ |
```